23/9/2026

Final এ minimum 10 mark অথবা 15 linked list

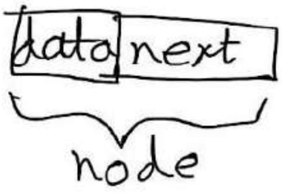
24/9/2026

Linked List

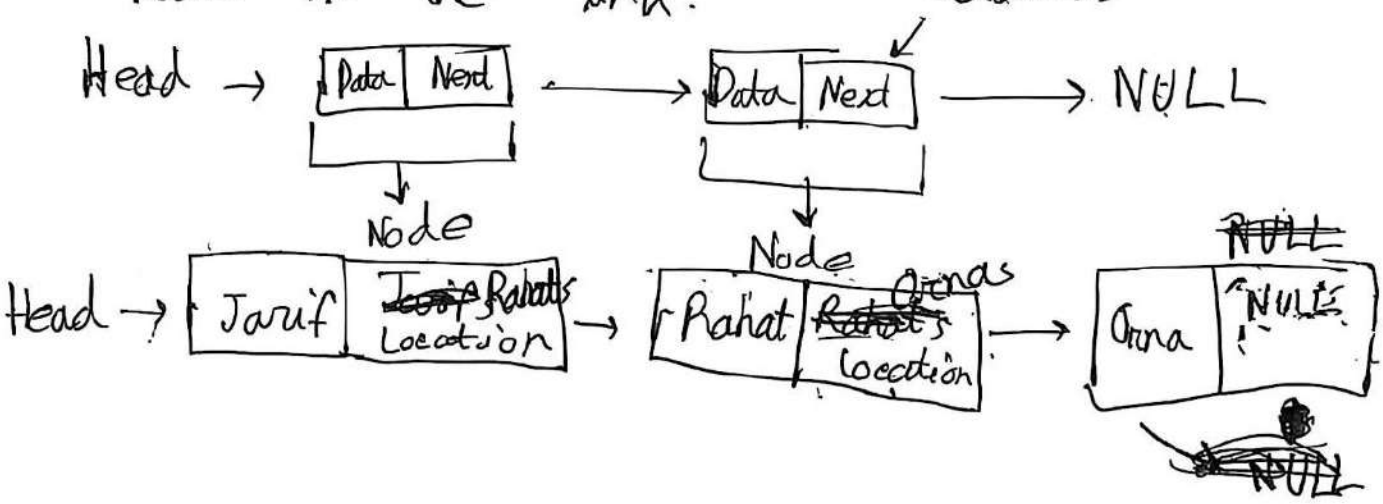
Linear data structure

- A linked list is an ordered collection of finite, homogeneous data elements called nodes where the linear order is maintained by means of links or pointers.

- Each element is represented by a node


data
next
node

- Each nod is divided into two parts.
  1. Data part contains the information of element
  2. Link part contains the address of the next node in the link.


address
Head
Data
Next
Data
Next
NULL
Node
Node
Head
Jarif
~~Jarif's~~ Rahat's Location
Rahat
~~Rahat's~~ Orna's Location
Orna
~~NULL~~
NULL
~~NULL~~

- Linked list has a pointer variable, Hear or start, containg adrress of first node
- If start contains NULL value, it means the list is empty
- The last node contains NULL represting invalid

Dynamic reprentation

Terminologies

Malloc()

```
#include <stdlib.h>
int* ptr = (int*) malloc (5*sizeof(int));
```

4 byte

(int*) optional

ptr = 20 bytes of memory

Structure

```
struct node
{
    int data;
    struct. ~~node~~ node *next;
};
```

Int নিজের ধারণ করে।

নিজের Pointer self refrence

5000 → 50[illegible]11

-> arrow operator

```
ptr = malloc (sizeof (struct node))
```

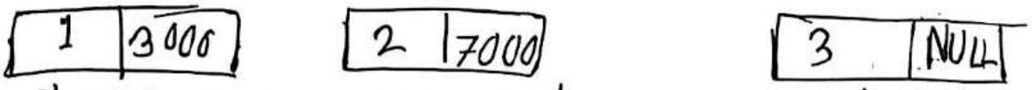
```
struct node *head;
struct node *one = NULL;
struct node *two = NULL;
struct node *three = NULL;

one = malloc (sizeof (struct node));
two = ";
three = "


one->data = 1;
two->data = 2;
three->data = 3;

one->next = two;
two->next = three;
three->next = NULL;

head = one;
```

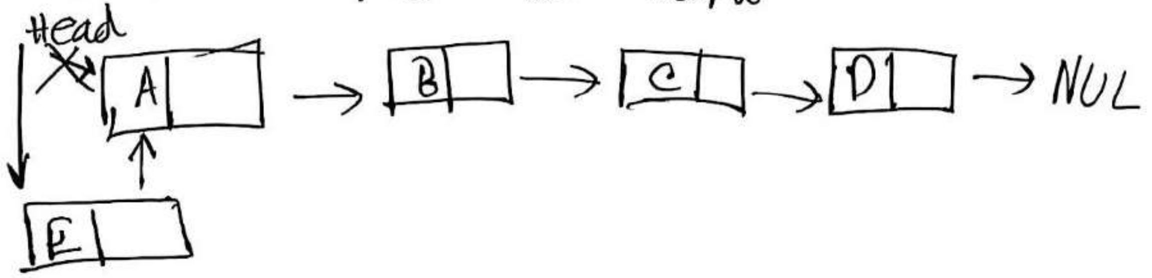


~~head =~~ 3000
~~head~~

head = 5000 *one = 5000 *second = 3000 *third = 7000

3409/2026

Add a new node at front

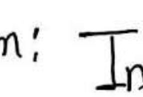

Head
A
B
C
D
NUL
E

Algorithm: InsertAtHead (head, data)

```
1. Create a new node NEW
2. Set NEW → data = data
3. Set NEW → next = head
4. Update head = NEW
5. Return head
```

Time compexity : O(1)

* head = NULL



*new = malloc( )   5000

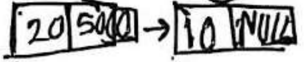
new → data = value;
10

new → next = head

head = new

---

① head = 5000

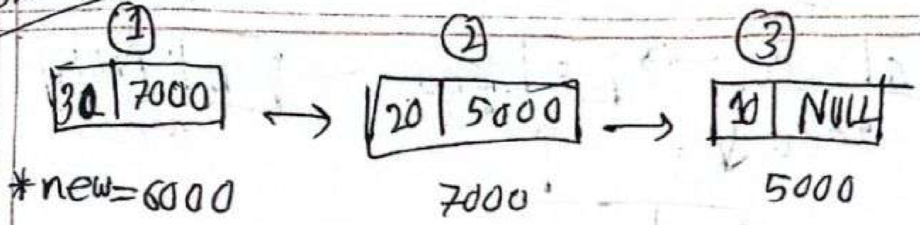

②
20
5000
10
NULL

*new = 7000

new → data = value = 20

new → next = head = 5000   head = new

3rd time calling

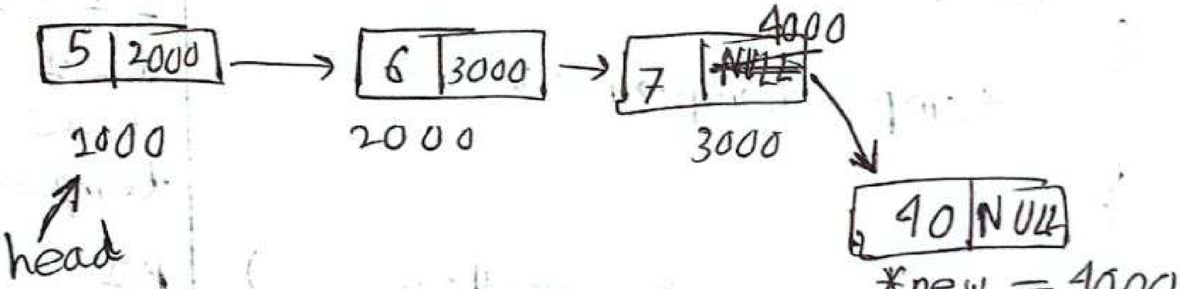

1
2
3
30 | 7000
20 | 5000
10 | NULL
*new=6000
7000
5000

new → data = value = 30

new → next = head = 7000

head = new

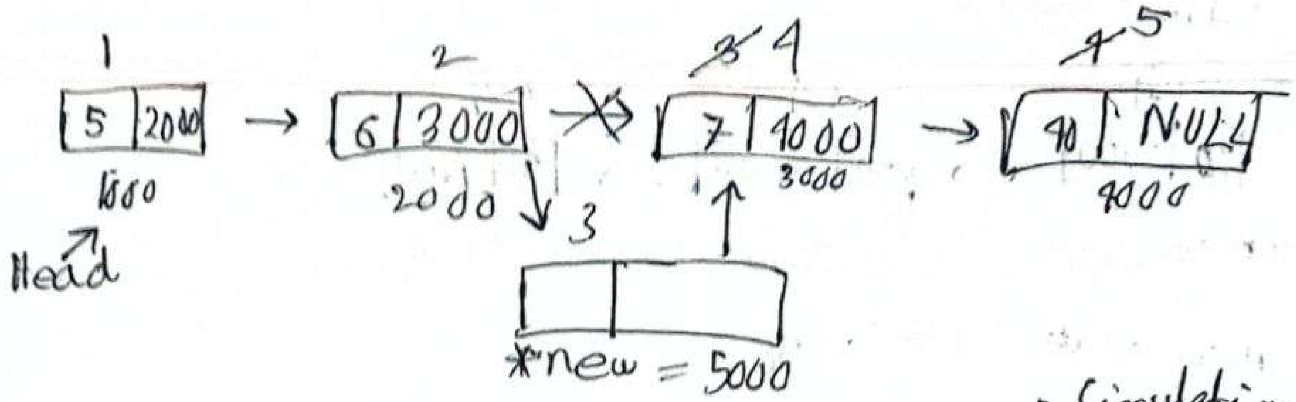
~~Algorithm~~: Insert at last


5 | 2000
6 | 3000
7 | ~~NULL~~ 4000
40 | NULL
1000
2000
3000
head
*new = 4000

```
struct node *temp = head;
while(temp→ next ! = NULL)
{
     temp = temp-> next;
}
```

লুপ শেষে temp Last node এ থাকবে.

```
struct node *new = malloc (........ )
            new -> data = value; // 40
            new -> next = NULL;
            temp → next = new;
```


1
2
~~3~~ 4
~~4~~ 5
5
2000
1000
6
3000
2000
7
1000
3000
10
NULL
4000
Head
3
*new = 5000

```
struct node *temp = head;
int pos;
scanf("%d", &pos);
for ( int i = 1; i < pos - 1; i++)
{
        temp = temp -> next;

}
struct node new* = malloc(---)
new -> data = value
new -> next = temp -> next
temp -> next = new
```

important lines

Simulation

temp = 1000

Round 1

i = 1

i < pos - 1

1 < 3 - 1

1 < 2

true

temp = 2000

R2

i = 2

2 < pos - 1

2 < 3 - 2

2 < 2

false

for loop আটকাবে

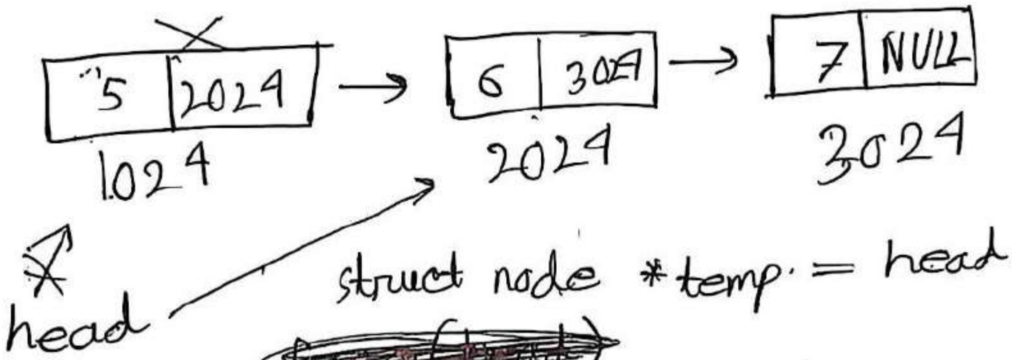
temp থাকবে 2000

01/10/26

free() function দিয়ে জায়গা খালি করে (Malloc এর উল্টা)

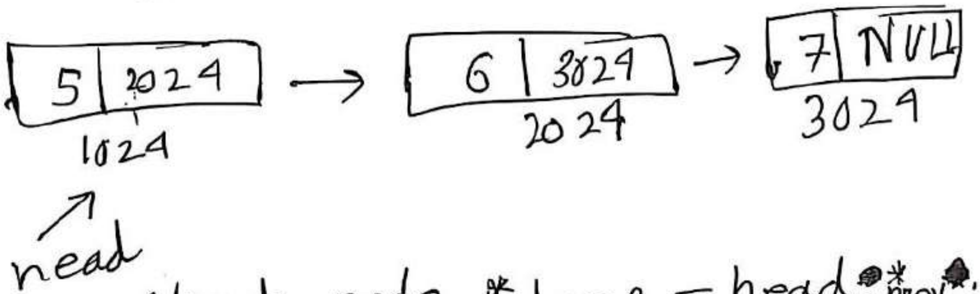
Deletion

Deletion from head


5
2024
1024
6
3024
2024
7
NULL
3024
head

```
struct node *temp = head; //temp=1024
~~free(head)~~
head = head -> next
free (temp);
```

Deletion from last -


5
2024
1024
6
3024
2024
7
NULL
3024
head

```
struct node *temp = head, *prev;
while (temp -> next != NULL) {
prev= temp; temp = temp -> next;
}
free (temp);
prev -> next = NULL;
```

Simulation

temp = 1024

R1

2024 != NULL

prev=1024;

temp = 2024

R2

3024 != NULL

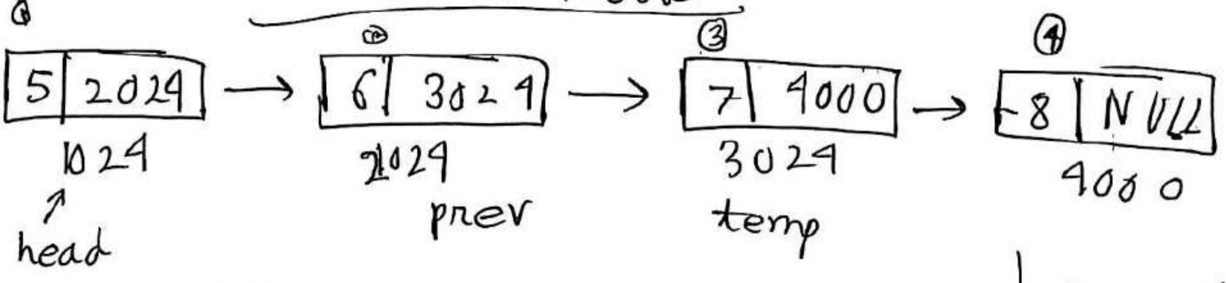
prev=2024;

temp = 3024

R3

NULL != NULL

Delete from middle


①
②
③
④
5 | 2024
6 | 3024
7 | 4000
8 | NULL
1024
2024
3024
4000
head
prev
temp

③ কে Delete করব

```
int pos;
scarf("%d", &pos);  // 3
struct node *temp = head, *prev;
for(int i=1;  i<pos;i++)
{      prev=temp;
      temp=temp->next;

 }
 prev->next = temp->next
 free(temp);
```

temp=1024
prev=?

R1

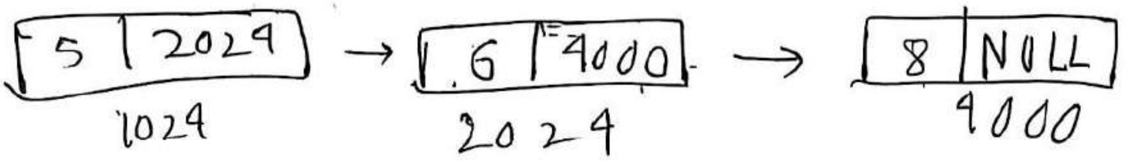
i=1
i<pos
1<3
prev=1024
temp=2024

R2

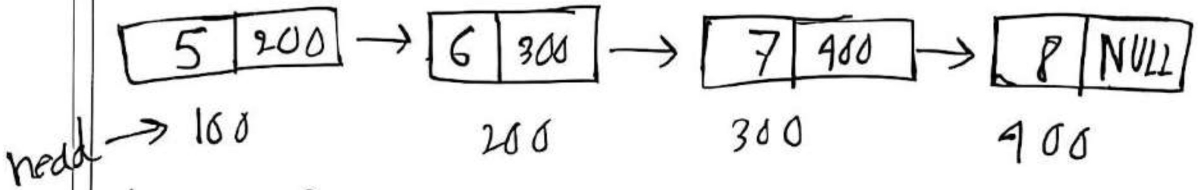
i=2
2<3
prev=2024
temp=3024

R3

i=3
3<3
✗


5 | 2024
6 | 4000
8 | NULL
1024
2024
4000

Search


5
200
6
300
7
400
8
NULL
head
100
200
300
400

key = 8

```
   int pos = 0
struct node *temp = head;
while(temp != NULL)
{        pos++;
      if (key == temp->data)
      {
          printf("Item found in pos %d", pos);

      }
      temp = temp->next;

}
```

temp = 100

R1

100 ! = NULL

8 = = ~~5~~

temp = 200

R2

200 ! NULL

8 ~~= =~~ 6

temp = 300

R3

300 != NULL

8 ~~= =~~ 7

temp = 400

R4

~~NULL~~ 400 != NULL

8 == 8

temp = NULL

R5 NULL != NULL

Circular singly Linkedlist

```
struct node *temp = head;
while (temp -> next ! = NULL)
{ temp = temp -> next;
}
temp -> next = head;
```

7/10/2026

ODP No. Node যুক্ত Linked list এর middle node delete

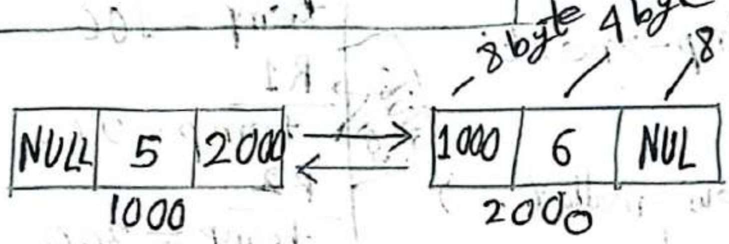
```
void delet middle ()
{ struct node *temp = head, *prev;
int  n=0;
   while(temp! = NULL) {
       n++;
      temp = temp -> next;
   }
  int m = n/2 + 1;
     temp = head;
  for(int i = 1; i < m; i++)
  {      prev = temp;
         temp = temp -> next;
  }
  prev -> next = temp -> next
  free (temp);
}
```

Doubly Linked List

prev	Data	Next

```
struct node{
    int data;
    struct node *next;
    struct node *prev;
};
```

Singly linke list - 12 byte
Doubly " " - 20 byte

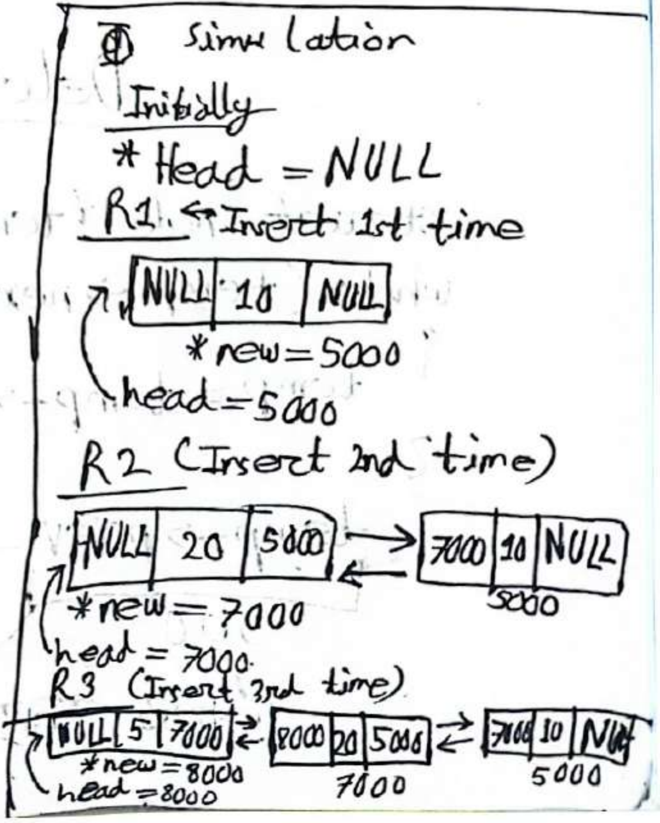

8 byte
4 byte
8 byte
NULL
5
2000
1000
1000
6
NUL
2000

Initially *head = NULL

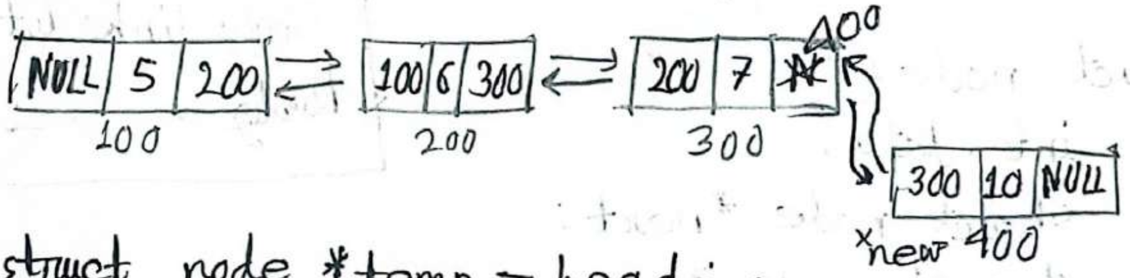
Insert at head

```
{
 struct. node *new = malloc(sizeof(struct node));
 new->data = value;
 new->next = head;
 new->prev = NULL;
if(head != NULL){
 head->prev = new;
 }
 head=new
}
```

[illegible]
[illegible] singly [illegible]


① Simulation
Initially
* Head = NULL
R1 ← Insert 1st time
NULL
10
NULL
*new=5000
head=5000
R2 (Insert 2nd time)
NULL
20
5000
7000
10
NULL
5000
*new = 7000
head = 7000.
R3 (Insert 3rd time)
NULL
5
7000
8000
20
5000
7000
10
NULL
*new=8000
head=8000
7000
5000

Insertion at Last


NULL
5
200
100
100
6
300
200
200
7
400
300
300
10
NULL
*new 400

```
struct node *temp = head;
while(temp->next != NULL)
{  temp = temp->next;

}
struct node *new=malloc(--)
new->data = value;
new->next = NULL;
new->prev = temp;
temp->next = new;
```

Singly থেকে শুধু
এই লাইনটাই extra

while loop

temp = 100

R1

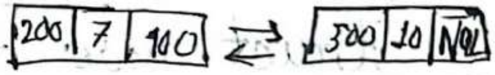
temp = 200

R2

temp = 300

R3

temp->next = NULL
Loop ends


200
7
400
300
10
NULL

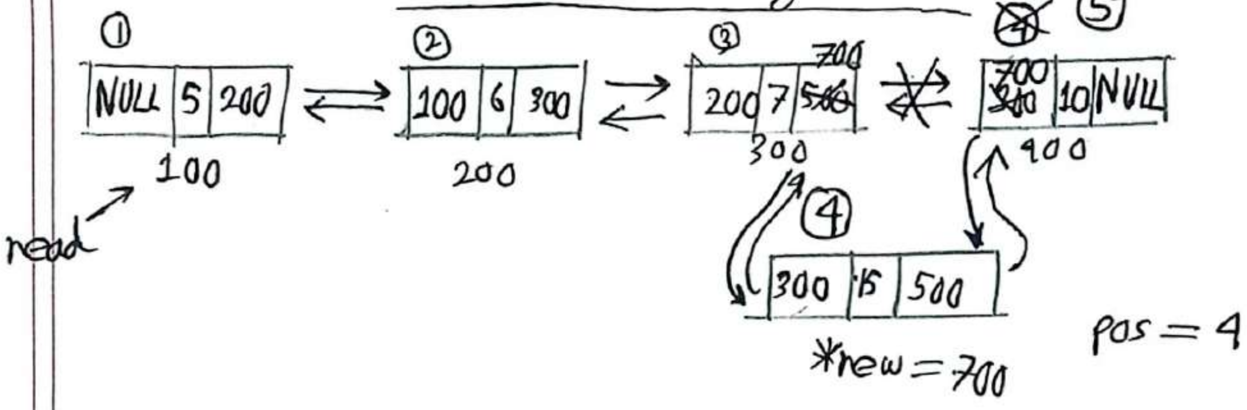
Delete at last

```
struct node *temp = head;
while (temp->next != NULL)
{
    temp = temp->next;
}
    temp->prev->next = NULL
    free(temp);
```

300

উপরের উদাহরণে

Insertion at any middle


①
②
③
~~④~~ ⑤
NULL 5 200
100 6 300
200 7 ~~500~~ 700
~~300~~ 700 10 NULL
100
200
300
400
head
④
300 15 500
*new = 700
pos = 4

```
struct node *temp = head;
for (int i = 1; i < pos - 1; i++)
{
    temp = temp -> next;
}
struct node *new = malloc (- - -)
new -> data = value;
new -> next = temp -> next;
new -> prev = temp ;
temp -> next = new ;
new -> next -> prev = new;
```

$\underbrace{\text{new -> next}}_{500}$

simulation

Loop ও temp = 100

R1
i = 1
temp = 200

R2
i = 2
temp = 300

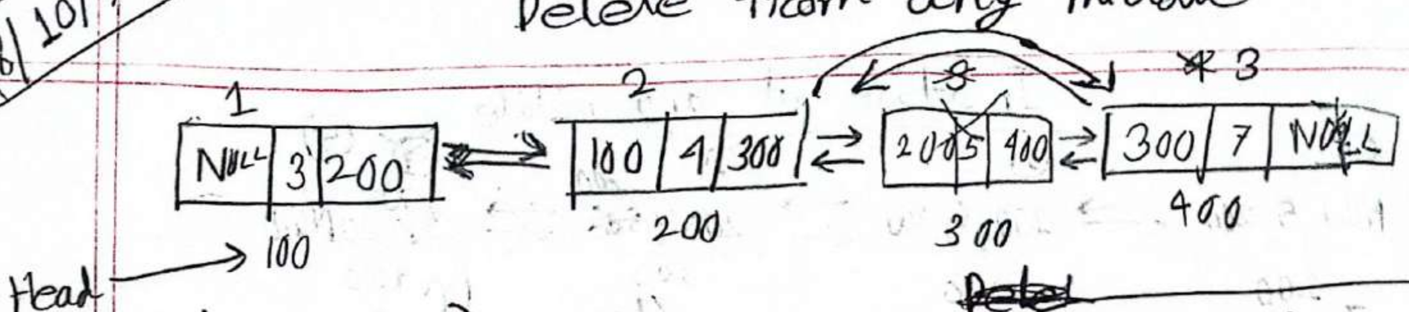
R3
i = 3
pos-1 = 4-1 = 3
3 ≮ 3
Loop ends

After loop, temp = 300

— × — × —

8/10/2026

Delete from any middle

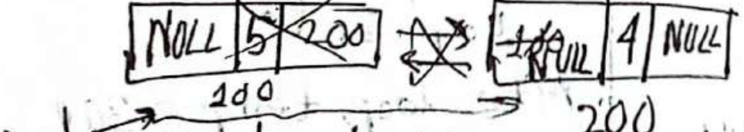

1
2
~~3~~
3
NULL
3
200
100
4
300
200
5
400
300
7
NULL
100
200
300
400
Head

Delete(3)

```
struct node *temp = head;
for(int i = 1; i<pos; i++)
{
    temp = temp->next;
}
temp->prev->next = temp->next;
200
temp->next->prev = temp->prev;
400
free(temp);
```

~~Delet~~
temp = 100
i = 1 < pos
R1 1 < 3
temp = 200
i = 2 < 3
R2
temp = 300
$i = 3 \not< 3$

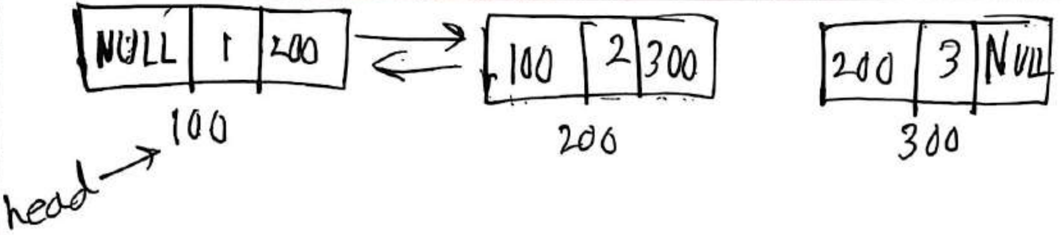
Delete from head


NULL
5
~~200~~
~~100~~ NULL
4
NULL
100
200
head

```
struct node *temp = head;   // temp = 100
head = head->next           //head = 200
head->prev = NULL;
free (temp);
```

Ring topology

Circular dobly linked list


NULL
1
200
100
2
300
200
3
NULL
100
200
300
head

```
struct node * temp = head;
while (temp -> next != NULL)
{  temp = temp -> next

}

temp -> next = head
head -> prev = temp
```

temp = 100

R1
temp = 200

R2
temp = 300

Last থেকে ~~[illegible]~~ উল্টা Loop

~~Ring topology~~

```
struct node *temp = head;
while (temp-> next != NULL){
        temp = temp->next;
}
while (temp != head){
    printf("%d", temp->data);
    temp = temp -> prev;
}
printf("%d", head->   data);
```

```
do {
printf("%d",temp->data);
temp=temp->prev;
while(temp!=head)
```

Solution by mam

```
struct node *temp =head;
while (temp -> next ! =NULL)
{ temp =temp -> next;
}

while ( temp! = NULL)
{ printf ("%d", temp->data);
  temp= temp -> prev;
}
```

Time complexity

Delet, Insert at head O(1)

Delete, Insertion at End O(n)

Delete, Insertion at specific location (Middle) O(n)

Useful

Space — Array তে আগে থেকে জায়গা রাখা লাগে। Linked list এ dynamicall জায়গা নেয় তাই সুবিধা